

Mindous.ai API Documentation

Version: 1.0.0

Base URL: `http://localhost:3000/api`

Authentication: Clerk (Bearer token in cookies)

Table of Contents

1. [Authentication](#)
 2. [Chat API](#)
 3. [Tasks API](#)
 4. [Analytics API](#)
 5. [Error Handling](#)
-

Authentication

All API endpoints require authentication via Clerk. The authentication token is automatically included in cookies when using the web interface.

Headers

```
Cookie: __session=<clerk_session_token>
```

Unauthorized Response

```
{  
  "error": "Unauthorized"  
}
```

Status Code: 401

Chat API

1. Create Chat Session

Endpoint: `POST /api/chat/sessions`

Description: Creates a new chat session for the authenticated user.

Request Body:

```
{
  "title": "My Chat Session" // Optional
}
```

Response:

```
{
  "id": "session_123abc",
  "userId": "user_456def",
  "title": "My Chat Session",
  "createdAt": "2025-11-17T15:00:00.000Z",
  "updatedAt": "2025-11-17T15:00:00.000Z"
}
```

Status Codes:

- 200 - Success
 - 401 - Unauthorized
 - 500 - Server error
-

2. Get Chat Sessions

Endpoint: GET /api/chat/sessions

Description: Retrieves all chat sessions for the authenticated user.

Query Parameters:

- limit (optional) - Number of sessions to return (default: 50)
- offset (optional) - Pagination offset (default: 0)

Response:

```
{
  "sessions": [
    {
      "id": "session_123abc",
      "userId": "user_456def",
      "title": "My Chat Session",
      "createdAt": "2025-11-17T15:00:00.000Z",
      "updatedAt": "2025-11-17T15:00:00.000Z",
      "messageCount": 5
    }
  ],
  "total": 10,
  "hasMore": true
}
```

Status Codes:

- 200 - Success
 - 401 - Unauthorized
 - 500 - Server error
-

3. Send Chat Message

Endpoint: POST /api/chat/message

Description: Sends a message to the AI and receives a streaming response.

Request Body:

```
{
  "sessionId": "session_123abc",
  "content": "Hello! Can you help me build a web app?"
}
```

Response: Server-Sent Events (SSE) stream

Stream Format:

```
data: {"content": "Hello", "type": "chunk"}
data: {"content": "! I'd", "type": "chunk"}
data: {"content": " be happy", "type": "chunk"}
data: {"type": "done"}
```

Status Codes:

- 200 - Success (streaming)
- 401 - Unauthorized
- 403 - LLM API error (missing API key)
- 404 - Session not found
- 500 - Server error

Notes:

- Requires ABACUSAI_API_KEY environment variable
- Messages are saved to database before and after streaming
- Recent conversation context (last 10 messages) is included

4. Get Chat Messages

Endpoint: GET /api/chat/sessions/:sessionId/messages

Description: Retrieves all messages for a specific chat session.

Path Parameters:

- sessionId - The chat session ID

Query Parameters:

- limit (optional) - Number of messages to return (default: 100)
- offset (optional) - Pagination offset (default: 0)

Response:

```
{
  "messages": [
    {
      "id": "msg_789ghi",
      "sessionId": "session_123abc",
      "role": "user",
      "content": "Hello!",
      "createdAt": "2025-11-17T15:00:00.000Z"
    },
    {
      "id": "msg_789ghj",
      "sessionId": "session_123abc",
      "role": "assistant",
      "content": "Hello! How can I help you today?",
      "createdAt": "2025-11-17T15:00:01.000Z"
    }
  ],
  "total": 10,
  "hasMore": false
}
```

Status Codes:

- 200 - Success
- 401 - Unauthorized
- 404 - Session not found
- 500 - Server error

Tasks API

1. Create Task

Endpoint: POST /api/tasks

Description: Creates a new task for the authenticated user.

Request Body:

```
{
  "title": "Build a todo app",
  "description": "Create a React-based todo application with authentication",
  "priority": "high", // Optional: "low", "medium", "high"
  "tags": ["react", "auth"] // Optional
}
```

Response:

```
{
  "id": "task_abc123",
  "userId": "user_456def",
  "title": "Build a todo app",
  "description": "Create a React-based todo application with authentication",
  "status": "pending",
  "priority": "high",
  "tags": ["react", "auth"],
  "createdAt": "2025-11-17T15:00:00.000Z",
  "updatedAt": "2025-11-17T15:00:00.000Z"
}
```

Status Codes:

- 200 - Success
 - 401 - Unauthorized
 - 400 - Invalid request body
 - 500 - Server error
-

2. Get Tasks

Endpoint: GET /api/tasks

Description: Retrieves all tasks for the authenticated user.

Query Parameters:

- status (optional) - Filter by status: "pending", "running", "completed", "paused", "failed"
- priority (optional) - Filter by priority: "low", "medium", "high"
- limit (optional) - Number of tasks to return (default: 50)
- offset (optional) - Pagination offset (default: 0)
- sortBy (optional) - Sort field: "createdAt", "updatedAt", "priority" (default: "createdAt")
- sortOrder (optional) - Sort order: "asc", "desc" (default: "desc")

Response:

```
{
  "tasks": [
    {
      "id": "task_abc123",
      "userId": "user_456def",
      "title": "Build a todo app",
      "description": "Create a React-based todo application",
      "status": "running",
      "priority": "high",
      "progress": 60,
      "subtaskCount": 5,
      "completedSubtasks": 3,
      "cost": 1.89,
      "createdAt": "2025-11-17T14:00:00.000Z",
      "updatedAt": "2025-11-17T15:00:00.000Z",
      "completedAt": null
    }
  ],
  "total": 15,
  "hasMore": false
}
```

Status Codes:

- 200 - Success
 - 401 - Unauthorized
 - 500 - Server error
-

3. Get Task by ID

Endpoint: GET /api/tasks/:taskId

Description: Retrieves a specific task with full details including subtasks.

Path Parameters:

- taskId - The task ID

Response:

```
{
  "id": "task_abc123",
  "userId": "user_456def",
  "title": "Build a todo app",
  "description": "Create a React-based todo application",
  "status": "running",
  "priority": "high",
  "progress": 60,
  "cost": 1.89,
  "subtasks": [
    {
      "id": "subtask_1",
      "title": "Set up React project",
      "status": "completed",
      "order": 1
    },
    {
      "id": "subtask_2",
      "title": "Implement authentication",
      "status": "running",
      "order": 2
    }
  ],
  "createdAt": "2025-11-17T14:00:00.000Z",
  "updatedAt": "2025-11-17T15:00:00.000Z"
}
```

Status Codes:

- 200 - Success
- 401 - Unauthorized
- 404 - Task not found
- 500 - Server error

4. Break Down Task

Endpoint: POST /api/tasks/breakdown

Description: Uses AI to break down a complex task into subtasks.

Request Body:

```
{
  "taskDescription": "Create a React todo app with authentication and deploy to Ver-  
cel"
}
```

Response:

```
{
  "subtasks": [
    {
      "title": "Initialize React project with TypeScript",
      "description": "Set up a new React project using Create React App with TypeScript template",
      "estimatedDuration": "15 minutes",
      "dependencies": []
    },
    {
      "title": "Implement authentication with Clerk",
      "description": "Add Clerk authentication provider and protect routes",
      "estimatedDuration": "30 minutes",
      "dependencies": ["subtask_1"]
    },
    {
      "title": "Build todo list components",
      "description": "Create components for adding, editing, and deleting todos",
      "estimatedDuration": "45 minutes",
      "dependencies": ["subtask_1", "subtask_2"]
    },
    {
      "title": "Deploy to Vercel",
      "description": "Configure Vercel deployment and deploy the application",
      "estimatedDuration": "20 minutes",
      "dependencies": ["subtask_3"]
    }
  ],
  "totalEstimatedDuration": "110 minutes",
  "complexity": "medium"
}
```

Status Codes:

- 200 - Success
- 401 - Unauthorized
- 403 - LLM API error (missing API key)
- 400 - Invalid request body
- 500 - Server error

Notes:

- Requires `ABACUSAI_API_KEY` environment variable
- Uses GPT-4.1-mini model for task decomposition

5. Update Task

Endpoint: `PATCH /api/tasks/:taskId`

Description: Updates a task's properties.

Path Parameters:

- `taskId` - The task ID

Request Body:


```
{
  "title": "Updated title", // Optional
  "description": "Updated description", // Optional
  "status": "paused", // Optional
  "priority": "low" // Optional
}
```

Response:

```
{
  "id": "task_abc123",
  "userId": "user_456def",
  "title": "Updated title",
  "description": "Updated description",
  "status": "paused",
  "priority": "low",
  "updatedAt": "2025-11-17T15:30:00.000Z"
}
```

Status Codes:

- 200 - Success
 - 401 - Unauthorized
 - 404 - Task not found
 - 400 - Invalid request body
 - 500 - Server error
-

6. Delete Task

Endpoint: DELETE /api/tasks/:taskId

Description: Deletes a task and all its subtasks.

Path Parameters:

- taskId - The task ID

Response:

```
{
  "success": true,
  "message": "Task deleted successfully"
}
```

Status Codes:

- 200 - Success
 - 401 - Unauthorized
 - 404 - Task not found
 - 500 - Server error
-

Analytics API

1. Get Task Analytics

Endpoint: GET /api/analytics/tasks

Description: Retrieves analytics data for the authenticated user's tasks.

Query Parameters:

- startDate (optional) - Start date for analytics (ISO 8601 format)
- endDate (optional) - End date for analytics (ISO 8601 format)
- groupBy (optional) - Group by: "day", "week", "month" (default: "day")

Response:

```
{
  "summary": {
    "totalTasks": 15,
    "completedTasks": 8,
    "runningTasks": 3,
    "pausedTasks": 2,
    "failedTasks": 2,
    "successRate": 0.8,
    "averageDuration": "45 minutes",
    "totalCost": 25.67
  },
  "timeline": [
    {
      "date": "2025-11-17",
      "tasksCreated": 5,
      "tasksCompleted": 3,
      "totalCost": 8.45
    }
  ],
  "byPriority": {
    "high": 6,
    "medium": 7,
    "low": 2
  },
  "byStatus": {
    "completed": 8,
    "running": 3,
    "paused": 2,
    "failed": 2
  }
}
```

Status Codes:

- 200 - Success
- 401 - Unauthorized
- 500 - Server error

2. Get LLM Usage Analytics

Endpoint: GET /api/analytics/llm

Description: Retrieves LLM usage statistics.

Query Parameters:

- `startDate` (optional) - Start date (ISO 8601 format)
- `endDate` (optional) - End date (ISO 8601 format)

Response:

```
{
  "totalRequests": 150,
  "totalTokens": 45000,
  "totalCost": 12.50,
  "byModel": {
    "gpt-4.1-mini": {
      "requests": 100,
      "tokens": 30000,
      "cost": 8.00
    },
    "claude-3-5-sonnet": {
      "requests": 50,
      "tokens": 15000,
      "cost": 4.50
    }
  },
  "averageResponseTime": "1.2s"
}
```

Status Codes:

- `200` - Success
- `401` - Unauthorized
- `500` - Server error

Error Handling

Standard Error Response

All API endpoints return errors in the following format:

```
{
  "error": "Error message",
  "code": "ERROR_CODE", // Optional
  "details": {} // Optional additional details
}
```

Common Error Codes

Status Code	Description
400	Bad Request - Invalid request body or parameters
401	Unauthorized - Missing or invalid authentication
403	Forbidden - Valid auth but insufficient permissions
404	Not Found - Resource doesn't exist
429	Too Many Requests - Rate limit exceeded
500	Internal Server Error - Server-side error
503	Service Unavailable - External service (LLM, DB) unavailable

Rate Limiting

- **Default Rate Limit:** 100 requests per minute per user
- **Chat API:** 20 requests per minute per user
- **Task Breakdown API:** 10 requests per minute per user

Rate Limit Headers:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1700236800
```

Rate Limit Exceeded Response:

```
{
  "error": "Rate limit exceeded",
  "retryAfter": 60
}
```

Environment Variables Required

LLM Configuration

```
ABACUSAI_API_KEY=your_abacus_api_key_here
OPENAI_API_KEY=your_openai_key_here
ANTHROPIC_API_KEY=your_anthropic_key_here
GOOGLE_API_KEY=your_google_key_here
```

Database

```
DATABASE_URL=postgresql://user:password@host:5432/database
```

Authentication

```
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_...  
CLERK_SECRET_KEY=sk_test_...
```

SDK Examples

JavaScript/TypeScript

```

// Create a chat session
const createSession = async () => {
  const response = await fetch('/api/chat/sessions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      title: 'My New Chat'
    }),
  });

  const session = await response.json();
  return session;
};

// Send a chat message with streaming
const sendMessage = async (sessionId: string, content: string) => {
  const response = await fetch('/api/chat/message', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      sessionId,
      content
    }),
  });

  const reader = response.body?.getReader();
  const decoder = new TextDecoder();

  while (true) {
    const { done, value } = await reader?.read() || { done: true };
    if (done) break;

    const chunk = decoder.decode(value);
    const lines = chunk.split('\n');

    for (const line of lines) {
      if (line.startsWith('data: ')) {
        const data = JSON.parse(line.slice(6));
        if (data.type === 'chunk') {
          console.log(data.content);
        }
      }
    }
  }
};

// Break down a task
const breakdownTask = async (description: string) => {
  const response = await fetch('/api/tasks/breakdown', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      taskDescription: description
    }),
  });
};

```

```
const breakdown = await response.json();
return breakdown;
};

// Get tasks with filters
const getTasks = async (status?: string) => {
  const params = new URLSearchParams();
  if (status) params.append('status', status);

  const response = await fetch(`/api/tasks?${params}`);
  const data = await response.json();
  return data.tasks;
};
```

Webhooks (Future Feature)

Webhook support for task completion and status updates is planned for a future release.

Changelog

Version 1.0.0 (2025-11-17)

- Initial API release
 - Chat API with streaming support
 - Tasks API with CRUD operations
 - Task breakdown with AI
 - Analytics API
 - Clerk authentication integration
-

Last Updated: November 17, 2025

Maintained By: Mindous.ai Development Team